

PRD v2

Product Requirements Document (PRD)

Metadata

- **Product Name:** my-fin
- **Version:** v1.1 (Refined for Local-First Sync Architecture)
- **Owner:** Sunil Kumar
- **Status:** Approved for Core Development

1. Executive Summary

my-fin is a privacy-first, offline-capable personal financial operating system designed to serve as the single source of truth for an individual's financial life. The application combines asset and liability tracking, zero-friction expense logging, bill reminders, credit card cycles, investment monitoring, vacation sub-ledgers, and predictive cash flow forecasting into a single unified workspace.

Unlike traditional platforms that force intrusive bank integrations or present purely retrospective spending graphs, my-fin focuses on future cash allocation boundaries and immediate financial clarity. Built on a single-codebase cross-platform architecture, it targets iOS, Android, and Web browsers seamlessly.

2. Problem Statement

Modern users manage their financial lives using highly fragmented, disconnected tools: single-purpose bank apps, credit card portals, investment dashboards, note-taking applications, static spreadsheets, and physical notebooks. This fragmentation causes serious operational issues:

- **Hidden Variables:** Forward-looking liabilities leak through the cracks, leading to recurring manual miscalculations.
- **Friction vs. Abandonment:** Traditional apps require invasive SMS or live bank scraping permissions, causing security anxiety. Manual apps fail because

logging everyday transactions becomes a tedious chore.

- **Retrospective Failure:** Existing platforms emphasize where money *went* last month via pie charts, failing to guide real-time spending choices today.

Core User Questions to Answer:

1. *How much clear, safe-to-spend money do I have right now?*
2. *What exact financial obligations are dropping over the next 15, 30, and 90 days?*
3. *Can I comfortably afford a specific planned asset purchase or vacation today?*

3. Vision & Product Goals

To establish a seamless personal financial environment that provides complete visibility into current liquidity, long-term asset bases, and short-term obligations while maintaining a fast, simple interaction framework optimized for daily entry.

- **Goal 1:** Deliver a unified real-time net worth and liquid cash interface.
- **Goal 2:** Eliminate manual tracking math via automated calculation layers and predictive algorithms.
- **Goal 3:** Prevent late fees, credit score damage, or missed cycles through deterministic notification matrices.
- **Goal 4:** Provide a safe sandbox environment to manage shared peer expenses and traveling ledgers without breaking core personal budgets.

4. Core Product Principles

- **Offline-First Resilience:** The application state must reside entirely on-device. The user interface must remain fully interactive, permitting edits, logs, and deletions with zero network connection.
- **Privacy-First Isolation:** User financial data is encrypted locally and isolated inside a secure silo. Data sync utilizes dedicated encrypted transport pipes with zero analytical cross-selling or monetization hooks.
- **Instant Entry Friction Lowering:** Creating a record must take under 3 seconds. Simple operations must not require navigating complex menus.

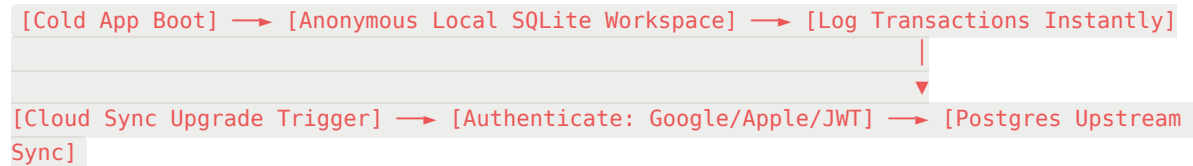
- **Forecast-Driven Execution:** The application prioritizes forward-looking cash flows over historical tracking, explicitly identifying future structural shortfalls before they manifest.

5. Target Audience Segments

- **Primary:** Engineers, contractors, and specialized professionals managing asymmetric income streams (salaries, consulting retainers, dividends), multiple high-utilization credit cards, and recurring investment plans.
- **Secondary:** Joint account holders, domestic couples, traveling groups, and peer groups requiring isolated sub-ledgers to split costs dynamically without polluting their personal master ledgers.

6. Authentication, Onboarding & Security Module

To eliminate drop-off friction, `my-fin` implements an un-gated onboarding model, deferring network authentication until cloud features are required.



6.1 Onboarding Architecture

- **Anonymous Local Sessions:** The application boots directly into a localized, interactive database shell. Users can build accounts, configure budgets, and record entries immediately without completing an email verification wall.
- **Account Sync Upgrade:** When a user opts into cloud backups, multi-device tracking, or shared workspaces, the app binds the local anonymous database records securely to a new authenticated cloud user profile.

6.2 Cryptographic Security

- **At-Rest Storage Encryption:** The client-side database layer is compiled with **SQLCipher (AES-256 bit encryption)**. The decryption keys are dynamically provisioned on-device and managed via hardware backends: **iOS Keychain** and **Android Keystore**.

- **Biometric Gating:** Native integration with FaceID, TouchID, and Android Biometrics. If enabled, the app enforces an immediate visual lock layer upon background-to-foreground state shifts or after 2 minutes of system inactivity.
- **Session Controls:** Short-lived JSON Web Tokens (JWT) manage authentication states. Users can view device logs and trigger a remote teardown command (`Logout from all active sessions`) to wipe local cryptographic keys instantly if a device is lost.

7. Account & Profile Customization Module

- **Profile Matrix:** Manages user preferences including name, email, global default currency symbols, localized timezone hooks, and specific financial start dates (e.g., matching cycles starting on the 1st or 5th of the month).
- **Account Structural Mapping:** Relational buckets managing money routing. Every account must carry the following attributes:

Attribute	Validation Rule	Purpose
<code>id</code>	UUID v4 generated on-client	Primary key for replication sorting.
<code>name</code>	String (Max 32 chars)	Clear label (e.g., "HDFC Savings").
<code>type</code>	Enum selection	<code>Bank</code> , <code>Cash</code> , <code>Digital Wallet</code> , <code>Credit Card</code> , <code>Investment</code> , <code>Loan</code> .
<code>balance</code>	Signed Decimal	Track absolute local values.
<code>credit_limit</code>	Unsigned Decimal (Nullable)	Required for Credit Card calculations.
<code>billing_day</code>	Integer 1-31 (Nullable)	Determines credit card statement generation boundaries.
<code>due_day</code>	Integer 1-31 (Nullable)	Triggers liability tracking timelines.

8. Income & Category Ledger Engines

- **Multi-Channel Input Routing:** Supports discrete classification of incoming cash streams (e.g., Base Salary, Retainers, Asset Dividends, Rental Revenue, P2P Transfers) to cleanly differentiate recurring income from one-off adjustments.

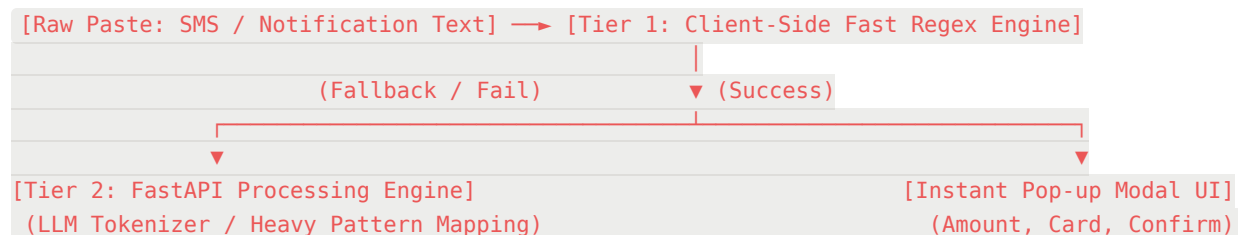
- **Category Trees:** A hierarchical tree separating core operations.
 - *System Defaults (Expenses):* Housing/Rent, Utilities, Groceries, Dining, Transport/Fuel, Entertainment, Insurance, Investments.
 - *System Defaults (Income):* Salary, Freelance, Investments, Adjustments.
 - *Custom Nodes:* Users can extend the category maps with custom child labels without breaking underlying parent analytical groups.

9. Transaction Management Engine

- **Atomic Mutations:** Every transaction is treated as an immutable accounting modification ledger. The system blocks open-ended updates, tracking history via adjustments to maintain database consistency.
- **Core Ledger Properties:** Records `id`, `amount`, `type` (`Income`, `Expense`, `Transfer`, `Refund`), `source_account_id`, `destination_account_id` (for transfers), `timestamp`, `tags`, and `workspace_id`.
- **Splits & Attachments:**
 - Users can divide a single master transaction into distinct sub-categories (e.g., a single invoice split into Groceries and Home Utilities).
 - Supports linking local file paths (receipt image metadata, invoice PDFs) stored securely within app sandboxes.

10. Core Smart Parser Pipeline

To enforce fast entry without compromising user privacy, the parsing framework operates as a two-tier decoupled system.



10.1 Tier 1: Client-Side Fast Regex Engine (MVP 1-2)

- **Execution:** Runs fully on-device within the React Native JavaScript loop.

- **Mechanics:** Scans clipboard contents against heavily optimized local pattern dictionaries matching standard Indian banking constructs (e.g., *"Debited for INR... at... via card..."* or UPI transfer strings).
- **Output:** Instantly extracts the `Amount` , `Merchant` , and `Source Account Reference` , populating a structured transaction form in under 100ms.

10.2 Tier 2: FastAPI Processing Engine (MVP 5)

- **Execution:** Cloud-based processing utility triggered when local pattern matching fails.
- **Mechanics:** Relies on an asynchronous FastAPI route running structured text parsing pipelines (via lightweight NLP classification models or regex dictionary maps).
- **Output:** Extracts complex parameters from unstructured email receipts or document OCR arrays, pushing clean payload responses back to the local device interface.

11. Real-Time Cash Flow Forecast & Budget Engine

- **The Soft-Ceiling Architecture:** Budgets operate as active warnings rather than rigid restrictions. Users configure macro target spend thresholds across categories, triggering alerts when velocity patterns indicate a category is tracking to exceed 80% of its budget before day 28.
- **The Net Liquidity Calculation:** The interface replaces standard retrospective asset tracking with a dynamic rolling liquidity score:

$$Net\ Liquidity = Total\ Cash\ \&\ Liquid\ Balances - Credit\ Card\ Outstanding_{[30\text{d}]} - Fixed\ Obligations_{[30\text{d}]}$$

- **Forecast Projections:** Generates horizontal timeline bars extending across **7, 15, 30, and 90-day windows**. By overlaying historical daily spending loops onto upcoming fixed liabilities, the engine calculates a rolling cash position graph and flags downstream balance shortfalls before they trigger.

12. Bills, Reminders & Credit Card Matrix

- **Chronological Timeline Engine:** Collects all future fixed commitments (EMIs, SIP schedules, recurring streaming subscriptions, phone recharges) and lists them inside a linear calendar view.
- **Notification Scheduling Matrix:** Dispatches deterministic native push alerts decoupled from external networks via local background workers using a multi-stage alert hierarchy:

[7 Days Before] → [3 Days Before] → [1 Day Before] → [Morning-Of Due Date]

- **Credit Card Utilization Monitor:** Tracks outstanding card debt against credit limits. If an individual card's utilization index passes 30%, the dashboard icon shifts state to signal an adverse credit utilization threshold.

13. Asset, Liability & Net Worth Engine

- **Asset Tracking Node:** Records illiquid or variable asset stores (Mutual Fund allocations, Stocks, Gold indexes, Employee Provident Fund, Real Estate holdings) separate from fluid transaction accounts.
- **Liability Core:** Tracks non-revolving long-term consumer debt profiles (Home loans, personal financing, automobile balances).
- **Net Worth Balance Calculation:** Updates macro valuation trends by evaluating the absolute accounting balance rule:

$$\text{Net Worth} = (\text{Liquid Accounts} + \text{Illiquid Assets}) - (\text{Credit Card Outstanding} + \text{Long-Term Liabilities})$$

- **Performance Analysis:** Tracks baseline metrics (\$XIRR\$, \$CAGR\$, asset valuation fluctuations) via manual balance revaluations or downstream batch updates, keeping illiquid portfolio calculations away from daily operational streams.

14. Personal Due Management Sub-Ledger

- **Peer Dynamics Mapping:** Tracks informal loans or shared group debts. Entries require an explicit relational direction indicator:
 - Owed to Me (Asset entry)
 - I Owe (Short-term operational liability)

- **Partial Settlement Chains:** Supports splitting a debt into a chain of partial ledger payments. A debt remains active across dashboards until its relational total reconciles to exactly zero.

15. Vacation Hub & Isolated Sandbox Modules

- **Siloed Ledgers:** Vacations function as isolated sub-ledgers. When initialized, all tracking parameters within the container operate independently from the main household budget.
- **Active Trip Mode:** Toggling a trip to "Active" shifts the app dashboard into a traveling view. The interface displays a daily trip budget, and all new transactions default directly into the trip sandbox.
- **Multi-Currency Offline Translator:** Stores local exchange rate vectors inside a device cache. When logging expenses offline (e.g., during international travel), the client records the local currency values while instantly updating estimates back to the base currency.

16. Shared Workspace Platform (Future Release - MVP 4)

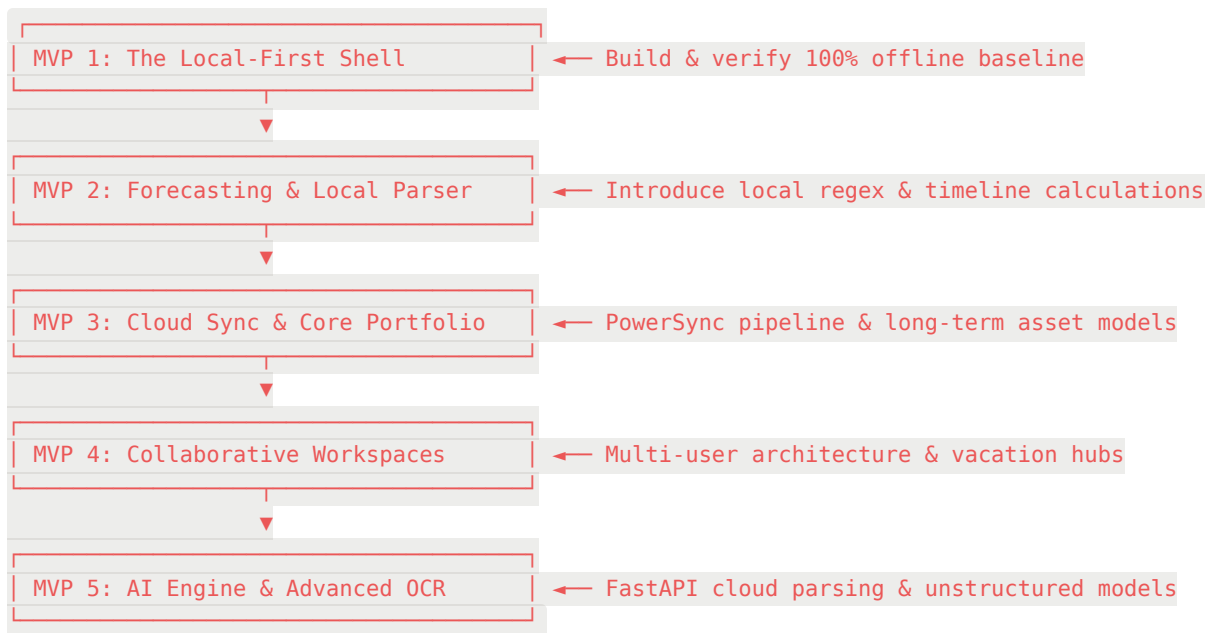
- **Workspace Isolation Architecture:** Transitions the app schema to support multi-tenant containers via dedicated workspace IDs. Personal entries utilize a private workspace partition, while collaborative pools utilize a shared database instance.
- **Replication Rule Filtering:** Employs an append-only transaction event log model. When devices reconnect to the cloud, the central server sequences the event timelines to prevent data overwrites.
- **Role-Based Access Controls (RBAC):** Restricts member read/write capabilities across workspaces based on three clear access groups:
 - **Owner** : Full write permission, account grouping capabilities, and deletion rights.
 - **Admin** : Full transaction modification and member invitation capabilities.
 - **Member** : Transaction creation and personal entry editing capabilities only.

17. Hardware Accelerated Analytics & UI

All charting and visualization modules bypass traditional DOM layout trees or mobile web wrappers, rendering directly onto a hardware-accelerated canvas via `@shopify/react-native-skia`.

- **Drill-Down Spending Donut:** A radial canvas component tracking category loads. Tapping a parent segment expands sub-category nodes via smooth spring transitions.
- **Cash Flow Burn Gauge:** A speedometer style interface mapping current consumption speeds. It calculates daily burn rates and warns if spending velocity will exhaust safe-to-spend limits prematurely.
- **Trend Performance Lines:** Multi-line graph structures mapping Income vs. Expense trajectories over 3, 6, and 12-month frames.

18. Structured MVP Implementation Roadmap



MVP 1: The Local-First Shell (Core Database & Entry)

- Focus: Establish the local application foundation. Build a fast, offline ledger that completely replaces physical notebooks or plain-text file tracking.

- **Database & Core:** Project scaffolding with Expo Router; local SQLite database integration compiled via SQLCipher for encryption-at-rest.
- **UI Modules:** Local anonymous onboarding shell, Account creation panels, Floating manual input triggers, and basic category selectors.
- **Dashboard:** Asset account list balances and a continuous historical transaction feed.

MVP 2: Proactive Forecasting & Clipboard Parsing

- Focus: Implement forward-looking timeline calculations and fast entry helpers.
- **Database & Logic:** Schema definitions for recurring transactions, liability reminders, credit card cycles, and peer dues.
- **Forecasting Engine:** Integration of the 30-day Net Liquidity Index and horizontal timeline graphs.
- **Fast Input:** Tier 1 Local Regex Parser module analyzing text strings from the user's clipboard.
- **Alert System:** Local background notification workers scheduling multi-stage payment alerts.

MVP 3: Cloud Synchronization & Macro Assets

- Focus: Bridge the local client data layer to secure cloud servers and track macro net worth.
- **Infrastructure:** Deployment of PostgreSQL core database (with logical replication enabled) and async FastAPI servers running authentication loops.
- **Sync Layer:** Integration of the PowerSync client SDK and local background data replication sync workers.
- **Advanced Ledger:** Portfolios modules tracking long-term assets, liabilities, and multi-currency exchange matrix caches.

MVP 4: Collaborative Workspaces & Travel Sandboxes

- Focus: Expand the architecture into collaborative multi-user environments.

- **Architecture Shift:** Migration of local data tables to support distinct Workspace IDs via append-only event streams.
- **Vacation Engine:** Sandbox trip containers, global trip toggles, and shared peer expense split matrices.
- **Access Control:** Server-side invite generations and token-based workspace claim verifications.

MVP 5: Intelligent Optimization & Advanced Capture

- Focus: Deep analytics, unstructured text ingestion, and pattern discovery.
- **AI Parser Pipeline:** Tier 2 cloud-driven document parsing and image OCR engines via FastAPI routes.
- **Analytics Engine:** Hardware-accelerated graphs using `@shopify/react-native-skia`.
- **Telemetry Insights:** Deployment of local telemetry queues tracking user retention patterns anonymously.

19. Success Metrics & Telemetry Framework

To preserve user privacy, `my-fin` completely bans persistent real-time tracker scripts or external analytical frameworks. Telemetry events use a localized buffer queue architecture.

- **The Local Analytics Queue:** Telemetry events (e.g., transaction record created, budget target exceeded, backup synchronization cycle completed) write directly to an isolated local database log.
- **Batch Background Processing:** When network connections are stable, the client compresses the event logs and uploads them in anonymous batches to a telemetry route on FastAPI. This avoids tracking user-identifiable financial values, focusing purely on functional system interaction speeds.

Key Performance Vectors:

- **Friction Free Ingestion Speed:** Target average entry time for clipboard parsing must remain below $< 1.5\text{s}$ from copy trigger to entry save.

- **Local UI Target Fluidity:** Interface rendering speeds across graphs and data matrices must sustain 60fps (16ms execution blocks) under heavy canvas rendering loops.
- **Sync Divergence Rate:** Unsynchronized or dropped mutation histories across multi-user environments must track below $< 0.1\%$ during reconnect transitions.